ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

«СИБИРСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ТЕЛЕКОММУНИКАЦИЙ И ИНФОРМАТИКИ»

ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине “Моделирование”

Выполнил студент Рудцких Владислав Евгеньевич

Ф.И.О.

Группы ИВ-222

Работу принял Ассистент кафедры ВС Уженцева А. В.

подпись

Защищена _____ Оценка

Новосибирск – 2026

Задание

Написать программу, которая позволит симулировать распределение элементов в трех ситуациях: Выборка с возвращением, выборка без возвращения, выборка с размещением.

Принцип работы

Выборка с возвращением и без

1) Генерируем или задаем числа m и n. Такие чтоб m был меньше или равен n

2) Заводим матрицу размерностью m x n, заполненную нулями

m - будет количеством элементов выборки

n - позиция в изначальном массиве

Пример работы:

n = 4. n = {1, 4, 2, 3}

из чисел из массива n рандомно выбираем числа для массива m

m = 3. m = {1, 3, 2}

На основе этого составить матрицу попаданий:

в данном примере в матрице 3 на 4 matrix[1][1] = 1, matrix[3][2] = 1, matrix[2][3] = 1, а все остальные значения по нулям

В итоге нужно вывести 12 матриц:

3 для выборки с возвращением (где в массиве m могут повторяться выдернутые числа из n)

3 для выборки без возвращения(где в массиве m не могут повторяться выдернутые числа из n)

3 для размещения - выборка без возвращения, где m==n

3 для выборки с несколькими прогонами, где значение элементов матрицы это число обозначающее количество попаданий выборки на эту позицию

Программная реализация

1. Генерация исходных данных (функция generateNArray**)**
Работа программы начинается с ввода пользователем двух ключевых параметров: n_size (размер исходного массива) и m (объем выборки). Функция generateNArray создает массив n_array заданного размера, заполняя его случайными числами от 1 до 9 с помощью вспомогательной функции getRandomInt. Этот массив символизирует генеральную совокупность, из которой будет производиться выборка. Значение m определяет, сколько строк будет в результирующих матрицах.

2. Выборка с возвращением (функция matrixWithReturn**)**
Данная функция моделирует схему, где каждый из m отобранных элементов может оказаться любым элементом исходного массива, независимо от предыдущих выборов.

- Создается матрица matrix размером m x n_size, изначально заполненная нулями.
- В цикле от 0 до m-1 для каждой строки генерируется случайный индекс index в пределах от 0 до n_size-1.
- В ячейку matrix[i][index] записывается 1, что означает, что элемент исходного массива с этим индексом был выбран в данной попытке. Поскольку индексы могут повторяться, в разных строках единицы могут стоять в одном и том же столбце, что иллюстрирует возвращение элемента.

3. Выборка без возвращения (функция samplingWithoutReplacement**)**
Эта функция реализует выборку, при которой один и тот же элемент не может быть выбран дважды.

- Создается вспомогательный список available_indexes, содержащий все индексы исходного массива.
- В цикле на m итераций выполняется выбор случайного индекса из available_indexes.
- Найденный индекс помечается единицей в соответствующей позиции строки матрицы, после чего этот индекс удаляется из списка available_indexes. Таким образом, для следующих строк он уже будет недоступен, гарантируя уникальность выбранных элементов в пределах одной матрицы.

4. Размещение (функция placement**)**
Данная функция является частным случаем выборки без возвращения, когда m равно n.

5. Многократные прогоны (функция multipleRuns**)**
Функция предназначена для наглядной демонстрации частоты при многократном повторении эксперимента с возвращением.

- Пользователь вводит количество прогонов runs.
- Создается матрица result_matrix размером m x n_size, заполненная нулями.

- В цикле run заданное количество раз вызывается функция matrixWithReturn. Результат каждого вызова поэлементно прибавляется к result_matrix. В итоговой матрице на пересечении строки и столбца будет стоять число, показывающее, сколько раз за все прогоны данный элемент исходного массива n_array был выбран в соответствующей попытке.

6. Визуализация результатов

Для наглядного представления данных используются две функции:

- printArray выводит сгенерированный исходный массив n_array в консоль.
- printMatrix форматирует вывод матриц, подписывая строки как m1, m2... (номера попыток), а столбцы как n1, n2... (индексы исходного массива), что позволяет легко интерпретировать результаты работы каждой функции.

Результат

Выборка с возвращением:

```
1. ВЫБОРКА С ВОЗВРАЩЕНИЕМ:

Пример 1:
    n1 n2 n3 n4 n5
m1  0  0  1  0  0
m2  0  1  0  0  0
m3  1  0  0  0  0

Пример 2:
    n1 n2 n3 n4 n5
m1  0  0  0  0  1
m2  0  1  0  0  0
m3  0  0  0  1  0

Пример 3:
    n1 n2 n3 n4 n5
m1  0  0  0  0  1
m2  1  0  0  0  0
m3  0  0  0  0  1
```

Выборка без возвращения:

```
2. ВЫБОРКА БЕЗ ВОЗВРАЩЕНИЯ:

Пример 1:
    n1 n2 n3 n4 n5
m1  0  1  0  0  0
m2  0  0  0  1  0
m3  0  0  1  0  0

Пример 2:
    n1 n2 n3 n4 n5
m1  0  0  0  0  1
m2  0  0  1  0  0
m3  0  1  0  0  0

Пример 3:
    n1 n2 n3 n4 n5
m1  0  1  0  0  0
m2  0  0  1  0  0
m3  1  0  0  0  0
```

Выборка с размещением:

```
3. РАЗМЕЩЕНИЕ (m = n, без возвращения):

Пример 1:
    n1 n2 n3 n4 n5
m1  0  0  1  0  0
m2  1  0  0  0  0
m3  0  1  0  0  0
m4  0  0  0  0  1
m5  0  0  0  1  0

Пример 2:
    n1 n2 n3 n4 n5
m1  1  0  0  0  0
m2  0  1  0  0  0
m3  0  0  0  1  0
m4  0  0  0  0  1
m5  0  0  1  0  0

Пример 3:
    n1 n2 n3 n4 n5
m1  0  1  0  0  0
m2  0  0  0  1  0
m3  1  0  0  0  0
m4  0  0  1  0  0
m5  0  0  0  0  1
```

Выборка с несколькими прогонами:

```
4. С НЕСКОЛЬКИМИ ПРОГОНАМИ:

Пример 1 (10 прогонов):
    n1 n2 n3 n4 n5
m1  3  1  4  1  1
m2  0  2  4  2  2
m3  1  2  2  4  1

Пример 2 (10 прогонов):
    n1 n2 n3 n4 n5
m1  3  1  1  2  3
m2  4  2  1  3  0
m3  1  4  2  3  0

Пример 3 (10 прогонов):
    n1 n2 n3 n4 n5
m1  3  2  2  2  1
m2  4  2  1  0  3
m3  1  0  2  3  4
```

Вывод

В ходе данной лабораторной работы подтверждены основные свойства различных типов случайных выборок. Матричное представление наглядно демонстрирует структуру выборки и позволяет легко анализировать распределение выбранных элементов. Особенно показательным является сравнение выборок с возвращением и без него, где четко прослеживается разница в допустимых комбинациях.

Приложение

```
#include <iostream>
#include <vector>
#include <random>
#include <algorithm>
#include <iomanip>

using namespace std;

void printMatrix(const vector<vector<int>>& matrix, const string& title) {
    cout << "\n" << title << ":\n";
    int m = matrix.size();
    int n = matrix[0].size();

    cout << "    ";
    for (int j = 0; j < n; j++) {
        cout << "n" << j+1 << " ";
    }
    cout << "\n";

    for (int i = 0; i < m; i++) {
        cout << "m" << i+1 << " ";
        for (int j = 0; j < n; j++) {
            cout << setw(2) << matrix[i][j] << " ";
        }
        cout << "\n";
    }
}


int getRandomInt(int min, int max) {
    static random_device rd;
    static mt19937 gen(rd());
    uniform_int_distribution<> dist(min, max);
    return dist(gen);
}


vector<int> generateNArray(int size) {
    vector<int> arr(size);
    for (int i = 0; i < size; i++) {
        arr[i] = getRandomInt(1, 9);
    }
    return arr;
}


void printArray(const vector<int>& arr, const string& name) {
    cout << name << " = {";
    for (size_t i = 0; i < arr.size(); i++) {
        cout << arr[i];
        if (i < arr.size() - 1) cout << ", ";
    }
    cout << "}\n";
}
```

```
vector<vector<int>> matrixWithReturn(const vector<int>& n, int m) {
    int n_size = n.size();
    vector<vector<int>> matrix(m, vector<int>(n_size, 0));

    for (int i = 0; i < m; i++) {
        int index = getRandomInt(0, n_size - 1);
        matrix[i][index] = 1;
    }

    return matrix;
}


vector<vector<int>> matrixWithoutReturn(const vector<int>& n, int m) {
    int n_size = n.size();
    vector<vector<int>> matrix(m, vector<int>(n_size, 0));

    vector<int> available_indexes(n_size);
    for (int i = 0; i < n_size; i++) {
        available_indexes[i] = i;
    }

    for (int i = 0; i < m; i++) {
        if (available_indexes.empty()) break;

        int random_pos = getRandomInt(0, available_indexes.size() - 1);
        int index = available_indexes[random_pos];
        matrix[i][index] = 1;


        available_indexes.erase(available_indexes.begin() + random_pos);
    }

    return matrix;
}


vector<vector<int>> placement(const vector<int>& n) {
    return matrixWithoutReturn(n, n.size());
}


vector<vector<int>> multipleRuns(const vector<int>& n, int m, int runs) {
    int n_size = n.size();
    vector<vector<int>> result_matrix(m, vector<int>(n_size, 0));

    for (int run = 0; run < runs; run++) {
        auto matrix = matrixWithReturn(n, m);
        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n_size; j++) {
                result_matrix[i][j] += matrix[i][j];
            }
        }
    }

    return result_matrix;
```

```
}

int main() {
    int n_size, m;
    cout << "Введите размер массива n: ";
    cin >> n_size;
    cout << "Введите размер выборки m (m <= n): ";
    cin >> m;

    if (m > n_size) {
        cout << "Ошибка: m должно быть <= n. Устанавливаю m = n.\n";
        m = n_size;
    }


    vector<int> n_array = generateNArray(n_size);
    cout << "\nСгенерированный массив:\n";
    printArray(n_array, "n");
    cout << "m = " << m << "\n\n";

    cout << "1. ВЫБОРКА С ВОЗВРАЩЕНИЕМ:\n";
    for (int i = 1; i <= 3; i++) {
        auto matrix = matrixWithReturn(n_array, m);
        printMatrix(matrix, "Пример " + to_string(i));
    }

    cout << "\n2. ВЫБОРКА БЕЗ ВОЗВРАЩЕНИЯ:\n";
    for (int i = 1; i <= 3; i++) {
        auto matrix = matrixWithoutReturn(n_array, m);
        printMatrix(matrix, "Пример " + to_string(i));
    }

    cout << "\n3. РАЗМЕЩЕНИЕ (m = n, без возвращения):\n";
    for (int i = 1; i <= 3; i++) {
        auto matrix = placement(n_array);
        printMatrix(matrix, "Пример " + to_string(i));
    }


    int runs;
    cout << "Введите количество прогонов: ";
    cin >> runs;

    cout << "\n4. С НЕСКОЛЬКИМИ ПРОГОНАМИ:\n";
    for (int i = 1; i <= 3; i++) {
        auto matrix = multipleRuns(n_array, m, runs);
        printMatrix(matrix, "Пример " + to_string(i) + " (" + to_string(runs) + "
прогонов)");
    }
    return 0;
}
```